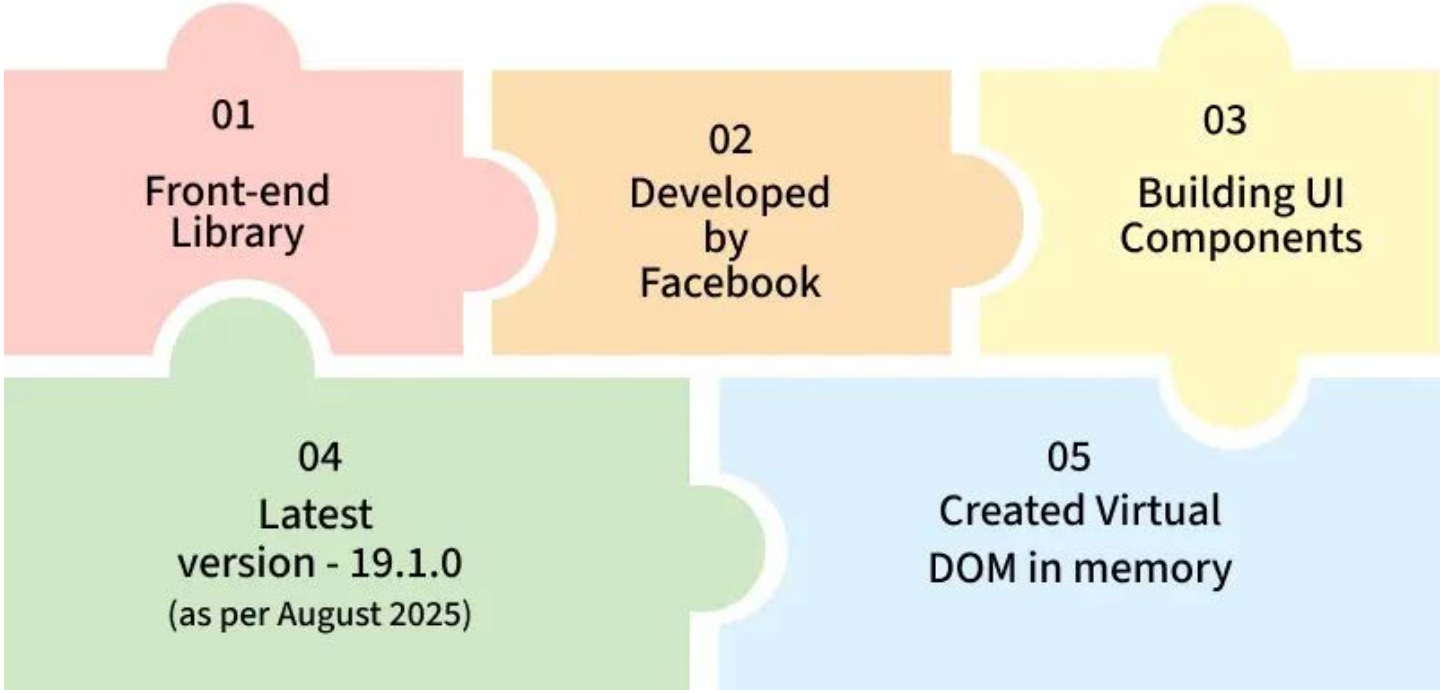
FULL-STACK APPLICATION DESIGN AND DEVELOPMENT

COURSE 3-4 - REACT

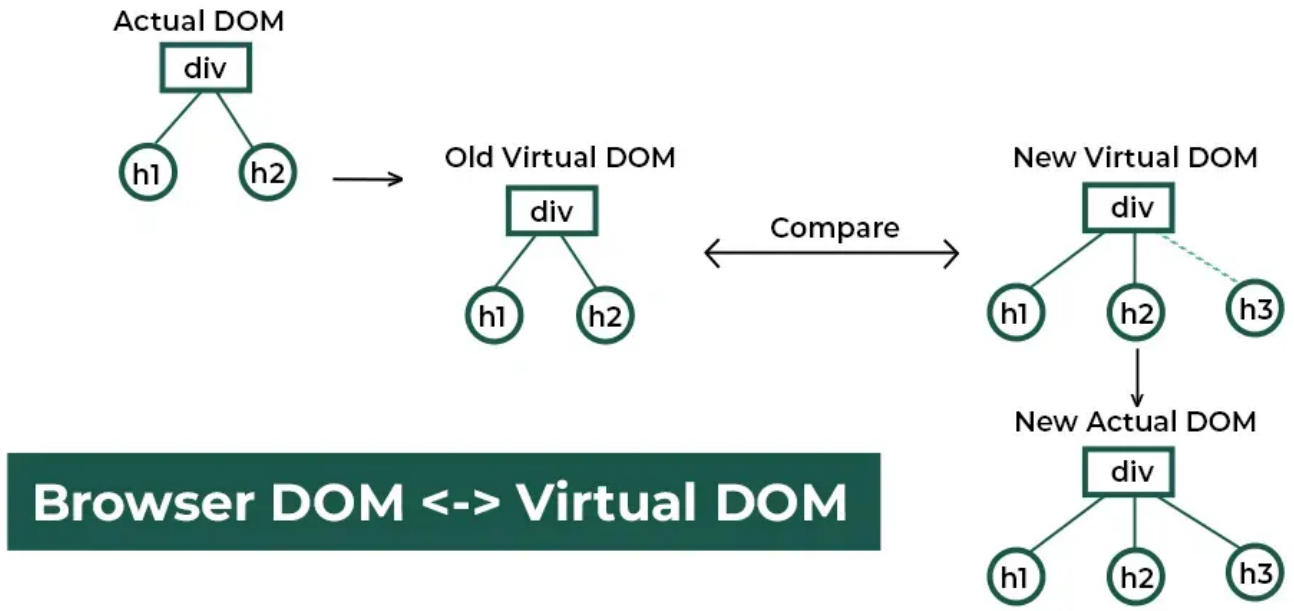
WHAT IS REACT?

- ReactJS is a **component-based JavaScript library** used to build dynamic and interactive user interfaces. It simplifies the creation of single-page applications (SPAs) with a focus on performance and maintainability.



HOW DOES REACT WORK?

- React operates by creating an in-memory virtual DOM rather than directly manipulating the browser's DOM. It performs necessary manipulations within this virtual representation before applying changes to the actual browser DOM.



HOW DOES REACT WORK?

Here's how the process works:

- **1. Actual DOM and Virtual DOM**
  - Initially, there is an Actual DOM (Real DOM) containing a div with two child elements: h1 and h2.
  - React maintains a previous Virtual DOM to track the UI state before any updates.
- **2. Detecting Changes**
  - When a change occurs (e.g., adding a new h3 element), React generates a New Virtual DOM.
  - React compares the previous Virtual DOM with the New Virtual DOM.
- **3. Efficient DOM Update**
  - React identifies the differences (in this case, the new h3 element).
  - Instead of updating the entire DOM, React updates only the changed part in the New Actual DOM, making the update process more efficient.

HOW DOES REACT WORK?

Feature	Real DOM	Virtual DOM
What is it?	The actual structure of a web page, a tree-like representation of HTML.	A lightweight, in-memory representation of the Real DOM.
Purpose	Represents the UI, allowing programs to access and modify content.	Optimizes performance by minimizing direct Real DOM manipulations.
Manipulation	Directly manipulates on-screen elements.	Does not directly manipulate on-screen elements; it's a diffing mechanism.
Performance	Can be slow for frequent updates as it re-renders the entire tree.	Fast updates due to diffing algorithm and batching of changes.
Encapsulation	No inherent encapsulation; styles and scripts are global	No inherent encapsulation; focuses on update efficiency.
Implementation	Native to web browsers.	Implemented by JavaScript libraries/frameworks (e.g., React, Vue).
Use Case	Fundamental for all web pages.	Used in modern JavaScript frameworks for efficient UI updates.
Direct UI Update	Yes, direct changes are immediately visible.	No, updates are first applied to the virtual copy, then "patched" to the Real DOM.
CSS Scoping	Global CSS, prone to conflicts.	Global CSS (unless handled by frameworks).

GETTING STARTED

Setting up React on Linux

- First, you need to install Node.js.
  - `sudo apt install nodejs`
- You can check if Node.js is installed by running the following command:
  - `node -v`
- Next, if npm (Node Package Manager) is not installed with Node.js, install it manually. This is the default package manager for Node.js, and it will be needed to install ReactJS and other dependencies.
  - `sudo apt install npm`
- You can check if npm is installed by running the following command:
  - `npm -v`

GETTING STARTED

- Once Node.js and npm are installed, you're ready to create your first React project.
- The recommended tool for this is **Vite**, which provides a fast and optimized setup.
- Vite is a modern build tool that offers a fast startup, minimal configuration, and a better development experience.
- Run this command to install Vite:
  - `npm install -g create-vite`

CREATE A REACT APPLICATION

- Next, run the following command to create a new React + JavaScript project using Vite:

```
npm create vite@latest my-app -- --template react
```

- Navigate to your project folder and install all required dependencies:

```
cd my-app
npm install
```

- Start the development server to preview your app in the browser:

```
npm run dev
```

- A new browser window will pop up with your newly created React App! If not, open your browser and type `localhost:5173` in the address bar.

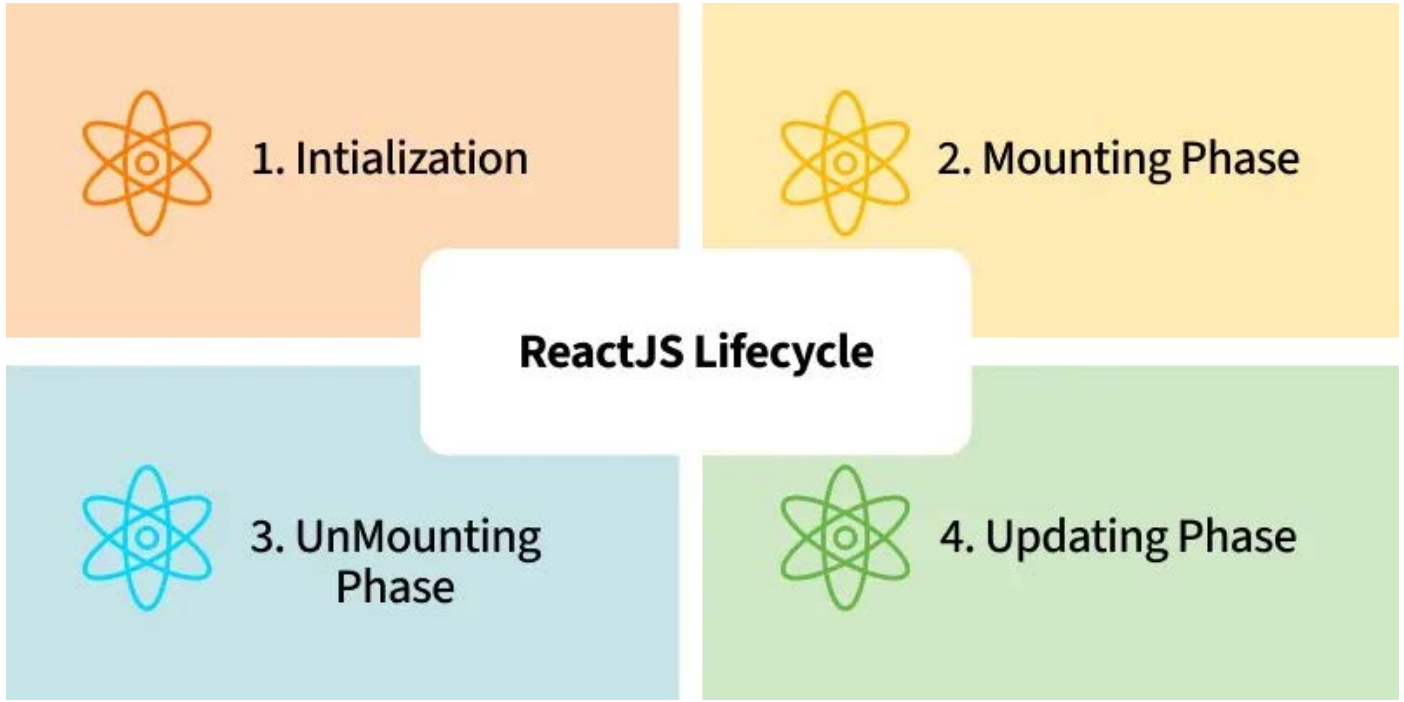
"HELLO, WORLD!" PROGRAM IN REACT

- Look in the `my-app` directory, and you will find a `src` folder. Inside the `src` folder there is a file called `App.jsx`, open it and try replacing the **entire file content** with the code below and save the file. The changes will be visible in the browser once you save the file.

```
import React from 'react';
function App() {
  return (
    <div>
      <h1>Hello, World!</h1>
    </div>
  );
}
export default App;
```

- In this example:
  - import React from 'react': Imports React to create components and use JSX.
  - function App() { ... }: Defines a functional component called App.
  - return (...): Returns JSX that represents the UI (a div with an h1 tag displaying "Hello, World!").
  - export default App: Exports the App component so it can be used elsewhere.

REACTJS LIFECYCLE



- Every React Component has a lifecycle of its own, the lifecycle of a component can be defined as the series of methods that are invoked in different stages of the component's existence.
- React automatically calls these methods at different points in a component's life cycle.
- Understanding these phases helps manage the state, perform side effects, and optimize components effectively.

REACTJS LIFECYCLE

1. Initialization
   - This is the stage where the component is constructed with the given Props and default state. This is done in the constructor of a Component Class.
2. Mouting Phase
   - **Constructor**: The constructor method initializes the component. It's where you set up the initial state and bind event handlers.
   - **render():** This method returns the JSX representation of the component. It's called during initial rendering and subsequent updates.
   - **componentDidMount():** After the component is inserted into the DOM, this method is invoked. Use it for side effects like data fetching or setting timers.

REACTJS LIFECYCLE

3. Updating Phase
   - **componentDidUpdate(prevProps, prevState)**: Called after the component updates due to new props or state changes. Handle side effects here.
   - **shouldComponentUpdate(nextProps, nextState):** Determines if the component should re-render. Optimize performance by customizing this method.
   - **render():** Again, the render() method reflects changes in state or props during updates.
4. Unmounting Phase
   - **componentWillUnmount()**: Invoked just before the component is removed from the DOM. Clean up resources (e.g., event listeners, timers).

APPLICATIONS OF REACT

- **Web Development:** React is used to build dynamic and responsive web applications, including social media platforms, e-commerce sites, and blogs.
- **Mobile Apps:** React Native allows developers to build mobile apps for iOS and Android using the same codebase.
- **Enterprise Applications**: React is used in building large-scale enterprise applications that require a highly interactive UI.
- **Dashboards and Data Visualizations:** React is great for building real-time dashboards and data visualization tools due to its high performance.

REACT IMPORTING AND EXPORTING COMPONENTS

- **In React**, components are the building blocks of any **application**.
- They allow developers to break down **complex user interfaces** into **smaller**, **reusable pieces**.
  - Importing and exporting components refer to sharing and using components across different files in your application.
  - Exporting a component means making it available for use in other files. Importing a component means bringing it into another file to use it.
  - JavaScript's import and export syntax allows you to bring in functions, objects, and components from one file into another, creating a modular and maintainable codebase.
  - React follows JavaScript's ES6 module system to handle imports and exports. You can have multiple components in separate files and import them when needed to make your code more modular and maintainable.

REACT IMPORTING AND EXPORTING COMPONENTS

- In React, there are two types of exports
  - Default Exports and Imports
  - Named Exports and Imports

1. **Default Export and Import**

- A default export allows you to export a single component or variable from a file. When importing a default export, you can give it any name you choose.

```
// app.js

import React from "react";
import MyComponent from "./components/MyComponent";

const App = () => {
    return (
        <div>
            <MyComponent /> {/* Using the imported
component */}
        </div>
    );
};
export default App;
```

```
// mycomponents.js

import React from "react";

const MyComponent = () => {
    return <h1>Hello from MyComponent!</h1>;
};

export default MyComponent;
```

REACT IMPORTING AND EXPORTING COMPONENTS

2. **Named Export and Import**

- Named exports allow you to export multiple components or variables from a single file. When importing a named export, you must use the exact name of the exported entity.

```
// app.js

import { MyComponent, AnotherComponent } from
"./components/component.js";

const App = () => {
    return (
        <div>
            <MyComponent />
            <AnotherComponent />
        </div>
    );
};

export default App;
```

```
// component.js

export const MyComponent = () => {
    return <h1>Hello from MyComponent!</h1>;
};

export const AnotherComponent = () => {
    return <h1>Hello from AnotherComponent!</h1>;
};
```

REACT IMPORTING AND EXPORTING COMPONENTS

- **When to Use Default Export**
  - Use for a single primary functionality or component in a file.
  - When you want flexibility in naming during import.
  - Ideal for components or modules that represent the main purpose of the file.
- **When to Use Named Export**
  - Use when exporting multiple functionalities or components from the same file.
  - When you want consistency in import names.
  - Useful for utility functions, constants, or multiple related components.

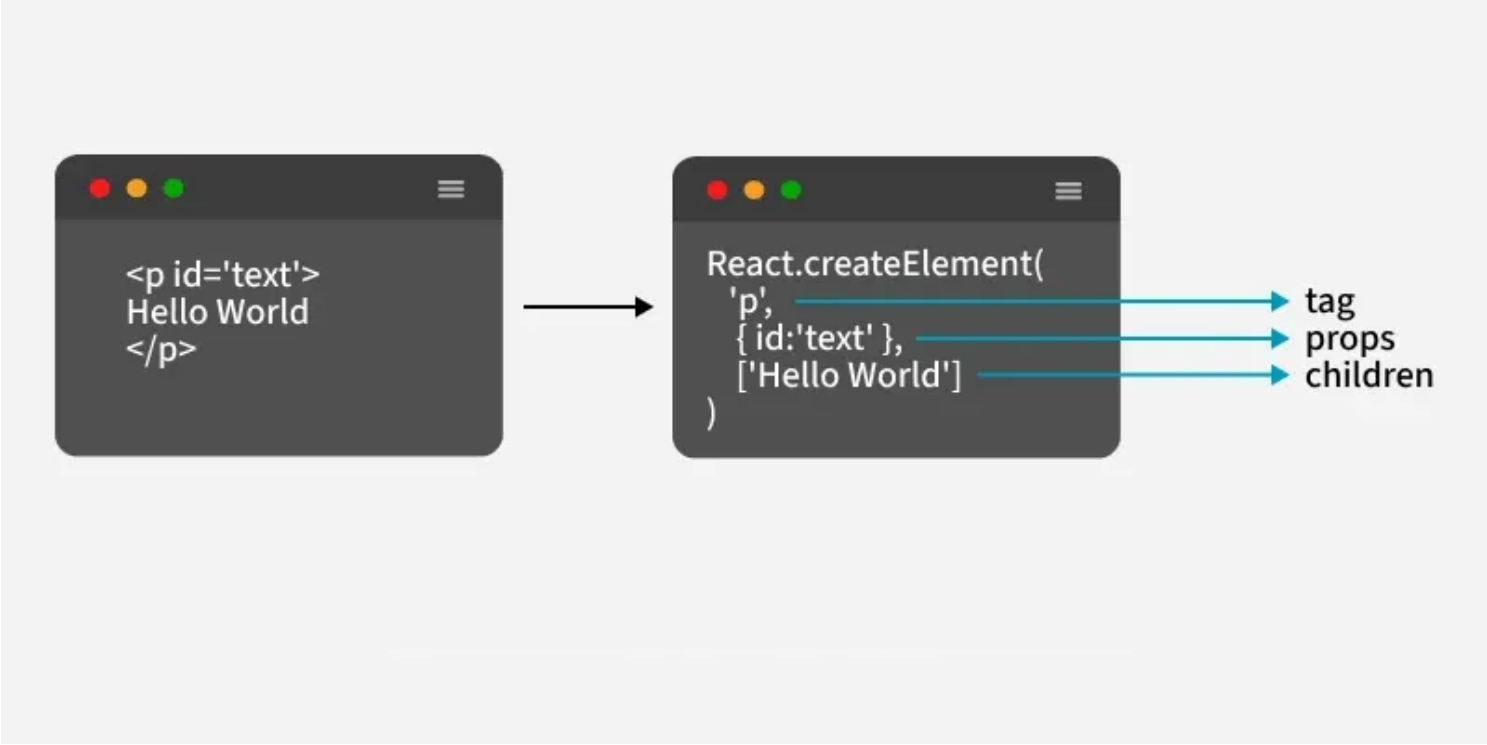
REACT JSX

- JSX stands for JavaScript XML, and it is a special syntax used in React to simplify building user interfaces.
- JSX allows you to write HTML-like code directly inside JavaScript, enabling you to create UI components more efficiently.
- Although JSX looks like regular HTML, it's actually a syntax extension for JavaScript.
- Example:

```
const element = <h1>Hello, world!</h1>;
```

REACT JSX

- When React processes this JSX code, it converts it into JavaScript using **Babel**. This JavaScript code then creates real HTML elements in the browser's DOM which is how your web page gets displayed.



REACT JSX

- **JSX Transformation Process**
- **Writing JSX:** Write JSX just like HTML inside JavaScript files (React components).

```
const element = <h1>Hello, World!</h1>;
```

- **JSX Gets Transformed:** JSX is not directly understood by browsers. So, it gets converted into JavaScript by a tool called **Babel**. After conversion, the JSX becomes equivalent to React.createElement() calls.

```
const element = React.createElement('h1', null, 'Hello, World!');
```

- **React Creates Elements:** React takes the JavaScript code generated from JSX and uses it to create real DOM elements that the browser can render on the screen.

REACT JSX

- JSX can be implemented in a React project to create dynamic and interactive UI components. Here are the steps to use JSX in a React application:
  - **Create a React App**
  - **Write JSX in the Component:** In the src/App.js file, write JSX to display a message:

```
import React from "react";
function App() {
    const message = "Hello, JSX works!";
    return <h1>{message}</h1>;
}
export default App;
```

  - The JSX code <h1>{message}</h1> will be transformed into JavaScript by Babel. Then, react will create a virtual DOM element for the <h1> tag with the text inside. This virtual DOM is used to update the actual browser DOM, displaying "Hello, JSX works!" on the screen.
  - After React processes the JSX, it renders the message on the screen.

REACT COMPONENTS

- In React, components are reusable, independent code blocks (a function or a class) that define the structure and behavior of the UI.
- They accept inputs (props or properties) and return elements that describe what should appear on the screen.
- **Key Concepts of React Components:**
  - Each component handles its own logic and UI rendering.
  - Components can be reused throughout the app for consistency.
  - Components accept inputs via props and manage dynamic data using state.
  - Only the changed component re-renders, not the entire page.

REACT COMPONENTS

```
import React from 'react';
// Creating a simple functional component
function Greeting() {
  return (
    <h1>Hello, welcome to React!</h1>
  );
}
export default Greeting;
```

- Greeting is a **React functional component**.
- It returns JSX: <h1>Hello, welcome to React!</h1>.
- ReactDOM.render mounts it to the DOM at an element with id root.

REACT COMPONENTS

- There are two primary types of React components:

1. **Functional Components**

- Functional components are simpler and preferred for most use cases. They are JavaScript functions that return React elements. With the introduction of React Hooks, functional components can also manage state and lifecycle events.
  - **Stateless or Stateful:** Can manage state using React Hooks.
  - **Simpler Syntax:** Ideal for small and reusable components.
  - **Performance:** Generally faster since they don't require a 'this' keyword.

```
function Greet(props) {
    return <h1>Hello, {props.name}!</h1>;
}
```

REACT COMPONENTS

2. **Class Components**

- Class components are ES6 classes that extend React.Component. They include additional features like state management and lifecycle methods.
  - **State Management:** State is managed using the this.state property.
  - **Lifecycle Methods:** Includes methods like componentDidMount, componentDidUpdate, etc.

```
class Greet extends React.Component {
    render() {
        return <h1>Hello, {this.props.name}!</h1>;
  }
}
```

REACT COMPONENTS

- **Props in React Components**
- Props (short for properties) are read-only inputs passed from a parent component to a child component. They enable dynamic data flow and reusability.
  - Props are immutable.
  - They enable communication between components.

```
function Greet(props) {
    return <h2>Welcome, {props.username}!</h2>;
}

// Usage
<Greet username=“John" />;
```

REACT COMPONENTS

- **State in React Components**
- The state is a JavaScript object managed within a component, allowing it to maintain and update its own data over time. Unlike props, state is mutable and controlled entirely by the component.
  - State updates trigger re-renders.
  - Functional components use the useState hook to manage state.

```
function Counter() {
    const [count, setCount] = React.useState(0);
    return (
        <div>
            <p>Count: {count}</p>
            <button onClick={() =>
                setCount(count + 1)}>Increment</button>
        </div>
    );
}
```

REACT COMPONENTS

- **Rendering a Component**
- Rendering a component refers to displaying it on the browser. React components can be rendered using the ReactDOM.render() method or by embedding them inside other components.
  - Ensure the component is imported before rendering.
  - The ReactDOM.render method is generally used in the root file.

```
ReactDOM.render(<Greeting name="Tom" />, document.getElementById('root'));
```

REACT COMPONENTS

- In React, you can nest components inside other components to build a modular and hierarchical structure.
  - Components can be reused multiple times within the same or different components.
  - Props can be passed to nested components for dynamic content.

```
function Header() {
    return <h1>Welcome to My Site</h1>;
}
function Footer() {
    return <p>© 2024 My Company</p>;
}
function App() {
    return (
        <div>
            <Header />
            <p>This is the main content.</p>
            <Footer />
        </div>
    );
}
export default App;
```

REACT COMPONENTS

- **Best Practices for React Components**
- **Keep Components Small:** Each component should do one thing well.
- **Use Functional Components:** Unless lifecycle methods or error boundaries are required.
- **Prop Validation:** Use PropTypes to enforce correct prop types.
- **State Management:** Lift state to the nearest common ancestor when multiple components need access.

REACT CONDITIONAL RENDERING

- Conditional rendering allows dynamic control over which UI elements or content are displayed based on specific conditions.
- It is commonly used in programming to show or hide elements depending on user input, data states, or system status.
- This technique improves user experience by ensuring that only relevant information is presented at a given time.
- It enables components to display different outputs depending on states or props.
- This ensures that the UI updates dynamically based on logic instead of manually manipulating the DOM.

REACT CONDITIONAL RENDERING

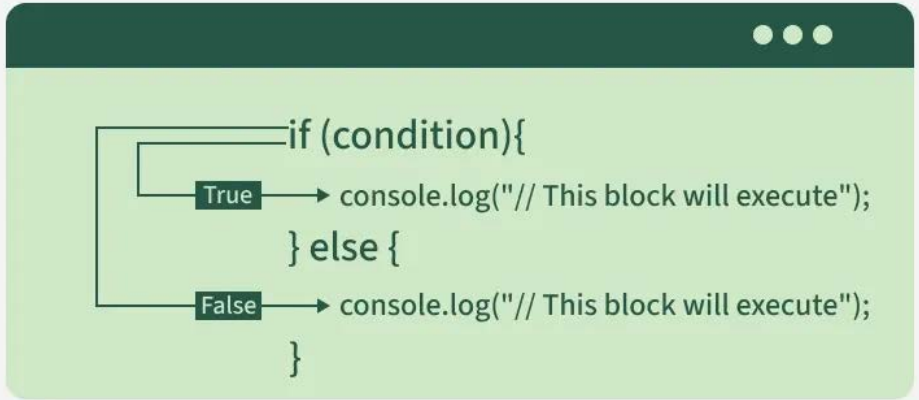
Implementation of Conditional Rendering

1. **Using If/Else Statements**

- If/else statements allow rendering different components based on conditions.
- This approach is useful for complex conditions.

```
function Item({ name, isPacked }) {
    if (isPacked) {
        return <li className="item">{name}</li>;

    }
    return <li className="item">{name}</li>;
}
```

- If isPacked is true, it displays: Space Suit
- If isPacked is false, it displays: Space Suit



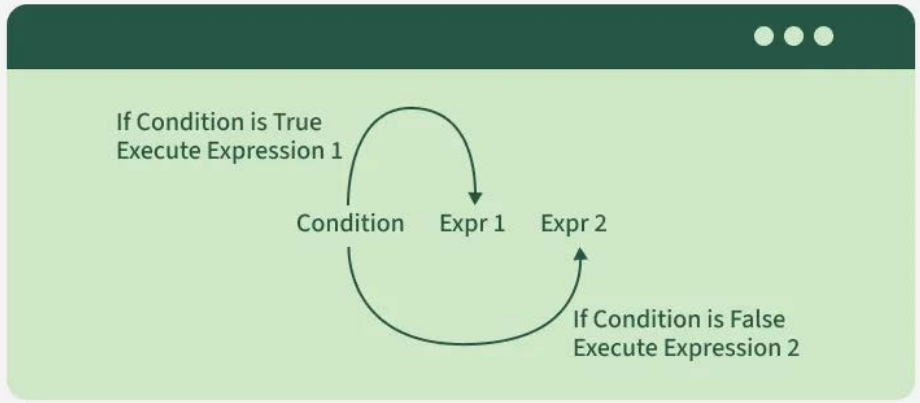
REACT CONDITIONAL RENDERING

2. **Using Ternary Operator**

- The ternary operator (condition ? expr1: expr2) is a concise way to conditionally render JSX elements.
- It's often used when the logic is simple and there are only two options to render.

```
function Greeting({ isLoggedIn }) {
    return <h1>{isLoggedIn ? "Welcome Back!" : "Please Sign In"}</h1>;
}
```

- If isLoggedIn is true: Welcome Back!
- If isLoggedIn is false: Please Sign In



REACT CONDITIONAL RENDERING

3. **Using Logical AND (&&) Operator**

- The && operator returns the second operand if the first is true, and nothing if the first is false.
- This can be useful when you only want to render something when a condition is true.

```
function Notification({ hasNotifications }) {
    return <div>{hasNotifications && <p>You have new
notifications!</p>}</div>;
}
```

- If hasNotifications is true: You have new notifications!
- If hasNotifications is false: Nothing is rendered.

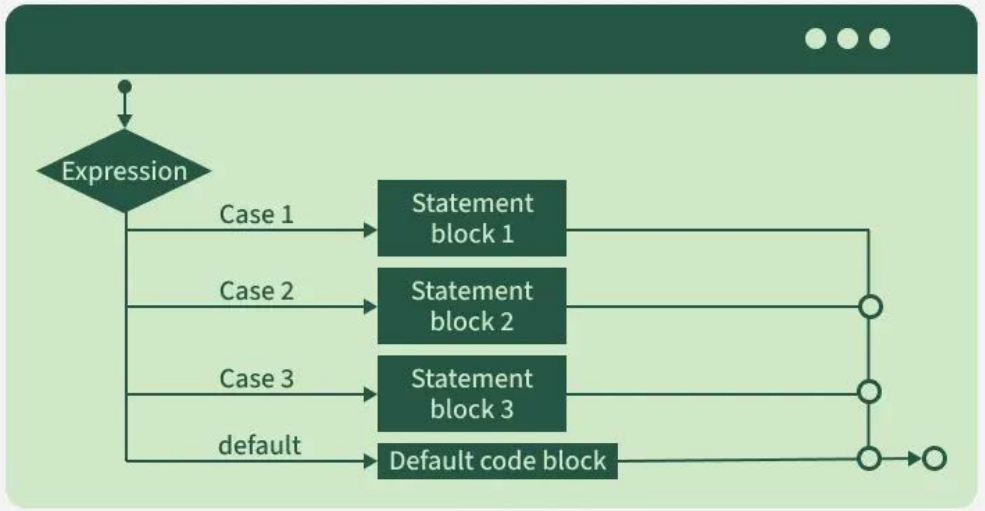
REACT CONDITIONAL RENDERING

4. **Using Switch Case Statements**

- Switch case statements are useful when you need to handle multiple conditions, which would otherwise require multiple **if** conditions. This approach can be more readable if there are many conditions to check.

```
function StatusMessage({ status }) {
    switch (status) {
        case 'loading':
            return <p>Loading...</p>;
        case 'success':
            return <p>Data loaded successfully!</p>;
        case 'error':
            return <p>Error loading data.</p>;
        default:
            return <p>Unknown status</p>;
    }
}
```

- If status is 'loading': Loading...
- If status is 'success': Data loaded successfully!
- If status is 'error': Error loading data.



REACT CONDITIONAL RENDERING

5. **Conditional Rendering in Lists (Using .map())**

- Conditional rendering can be helpful when rendering lists of items conditionally.
- You can filter or map over an array to selectively render components based on a condition.

```
const items = ["Apple", "Banana", "Cherry"];
const fruitList = items.map((item, index) =>
    item.includes("a") ? <p key={index}>{item}</p> : null
);
```

- If the item contains letter "a", it will be displayed.

REACT LISTS

- To render a list in React, we will use the JavaScript array map() function. We will iterate the array using map() and return the required element in the form of JSX to render the repetitive elements.
- The key prop is essential in React when rendering lists because it helps to identify which items have changed, been added, or removed. This allows React to efficiently update and re-render only the changed items in the DOM, rather than re-rendering the entire list.

```
function App() {
    const items = ['Apple', 'Banana', 'Cherry'];
    return (
        <div>
            <h1>My Fruit List</h1>
            <ul>
                {items.map((item, index) => (
                    <li key={index}>{item}</li>
                ))}
            </ul>
        </div>
    );
}
export default App;
```

REACT LISTS

List with Objects

- Lists can also be created using objects where each item contains multiple properties.

```
const users = [
    { id: 1, name: ‘John', age: 30 },
    { id: 2, name: ‘Mary', age: 25 },
    { id: 3, name: ‘Sam', age: 20 },
];
const App = () => {
    return (
        <ul>
            {users.map((user) => (
                <li key={user.id}>
                    {user.name} is {user.age} years old.
                </li>
            ))}
        </ul>
    );
}
export default App;
```

REACT LISTS

Conditional Rendering in Lists

- Sometimes, you may want to render items conditionally based on certain conditions.

```
const App = () => {
    const users = [
        { id: 1, name: 'John', age: 30 },
        { id: 2, name: 'Mary', age: 25 },
        { id: 3, name: 'Sam', age: 35 },
    ];
    return (
        <ul>
            {users.map((user) => (
                user.age > 30 ? (
                    <li key={user.id}>{user.name} is over 30 years old</li>
                ) : (
                    <li key={user.id}>{user.name} is under 30 years old</li>
                )
            ))}
        </ul>
    );
};
export default App;
```

REACT LISTS

- A "List with a Click Event" in React typically refers to displaying a list of items (such as an array) on the screen, where each item is associated with an event handler (such as onClick) that executes a function when the item is clicked.

```
const App = () => {
    const COMPANY = [“BEN", “AND", “JERRY’S"];
    const handleClick = (COMPANY) => {
        alert(`You clicked on ${COMPANY}`);
    };
    return (
        <ul>
            {COMPANY.map((COMPANY, index) => (
                <button key={index} onClick={() => handleClick(COMPANY)}>
                    {COMPANY}
                </button>
            ))}
        </ul>
    );
};
export default App;
```

CONTEXT API

- Context API in React is used to share data across the components without passing the props manually through every level. It helps when there is a need for sharing state between a lot of nested components.
- To work with Context API we need React.createContext(). It has two properties Provider and Consumer.
  - **Provider:** This is used to provide the context to components.
  - **Consumer:** This is used to consume the context value in child components.

CONTEXT API

- **Steps to Implement Context API in React**
- **Step 1:** Create a React application using the following command

  ```
  npx create-react-app context-api-demo
  ```

- **Step 2:** After creating your project folder, move to it by using the following command

  ```
  cd context-api-demo
  ```

- **Step 3:** We will create two new files, one is *Context.js* to create our context and another file, *WelcomePage.js* for creating a component that will consume the context.

CONTEXT API

- We will use Context API to display a user's name and ID.
- First, go to the `Context.js` file in the `src` folder, where we define the `UserContext.` The context provides `Provider` and `Consumer`, so we'll store them in constants.
- In a simple component file, we'll display a message showing the user's name and ID, which will be passed through the `Provider`.
- Finally, wrap the `App` component in `index.js` with the `Provider` and pass the name and ID as values. If no value is passed, the page will be blank.
- Your project will have the following files:
  - **Context.js:** We create the consumer and provider in this file
  - **WelocomePage.js:** The consumer consumes the value in this file
  - **Index.js:** The provider is given to the application in this file
  - **App.js:** The components are imported in this file and then rendered on the webpage

CONTEXT API

```
// Context.js
import React, { createContext } from 'react';
export const UserContext = createContext();

const UserProvider = ({ children }) => {
  const user = { name: 'John Doe', id: 1 };

  return (
    <UserContext.Provider value={user}>
      {children}
    </UserContext.Provider>
  );
};

export default UserProvider;
```

CONTEXT API

```
// WelcomePage.js
import React, { useContext } from 'react';
import { UserContext } from './Context';

const WelcomePage = () => {
  const { user } = useContext(UserContext);
  return (
    <div>
      <h1>Welcome User:</h1>
      <p>Name: {user.name} id: {user.id}</p>
    </div>
  );
};

export default WelcomePage;
```

CONTEXT API

```
// index.js
import React from 'react';
import ReactDOM from 'react-dom';
import './index.css';
import App from './App';

ReactDOM.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
  document.getElementById('root')
);
```

CONTEXT API

```
// App.js
import React from 'react';
import WelcomePage from './WelcomePage';
import UserProvider from './Context';

const App = () => {
  return (
    <UserProvider>
      <WelcomePage />
    </UserProvider>
  );
};

export default App;
```

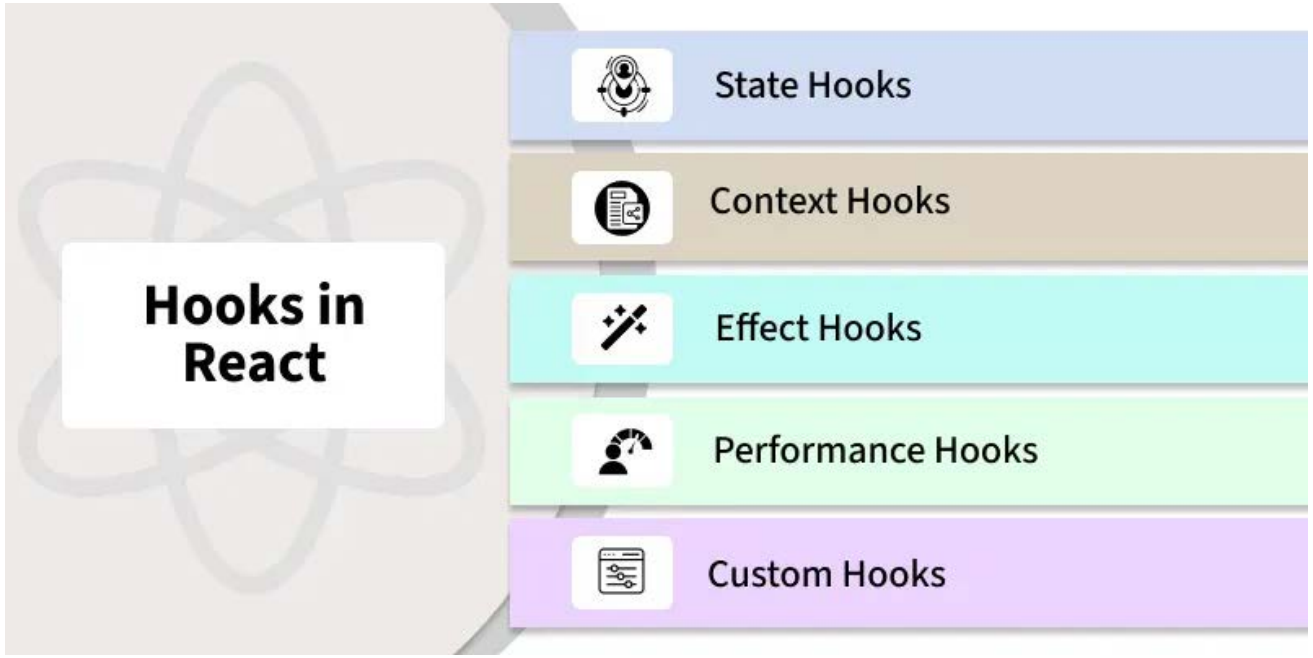
REACT HOOKS

- Hooks allow functions to have access to state and other React features without using classes.
- They provide a more direct API to React concepts like props, state, context, refs, and lifecycle.
- Here is an example of a Hook:

```
import { useState } from 'react';
import { createRoot } from 'react-dom/client';

function FavoriteColor() {
  const [color, setColor] = useState("red");

  return (
    <>
      <h1>My favorite color is {color}!</h1>
      <button
        type="button"
        onClick={() => setColor("blue")}
      >Blue</button>
```

```
<button
        type="button"
        onClick={() => setColor("red")}
      >Red</button>
      <button
        type="button"
        onClick={() => setColor("pink")}
      >Pink</button>
      <button
        type="button"
        onClick={() => setColor("green")}
      >Green</button>
    </>
  );
}

createRoot(document.getElementById('root')).render(
  <FavoriteColor />
);
```

REACT HOOKS



- React offers various hooks to handle state, side effects, and other functionalities in functional components.
- Some of the most commonly used types of React hooks are: state hooks, context hooks, effect hooks, performance hooks, custom hooks, etc.
- There are 3 rules for hooks:
  - Hooks can only be called inside React function components.
  - Hooks can only be called at the top level of a component.
  - Hooks cannot be conditional.

REACT HOOKS – STATE HOOKS

- State hooks, specifically `useState` and `useReducer`, allow functional components to manage state in a more efficient and modular way.
- They provide an easier and cleaner approach to managing component-level states in comparison to class components.
- `useState:` The `useState` hook is used to declare state variables in functional components. It allows us to read and update the state within the component.

```
const [state, setState] = useState(initialState);
```

- `state:` The current value of the state.
- `setState:` A function used to update the state.
- `initialState:` The initial value of the state, which can be a primitive type or an object/array.

REACT HOOKS – STATE HOOKS

- `useReducer:` The `useReducer` hook is a more advanced state management hook used for handling more complex state logic, often involving multiple sub-values or more intricate state transitions.

```
const [state, dispatch] = useReducer(reducer, initialState);
```

- `state:` The current state value.
- `dispatch:` A function used to dispatch actions that will update the state.
- `reducer:` A function that defines how the state should change based on the dispatched action.
- `initialState:` The initial state value.

REACT HOOKS – STATE HOOKS

```
import { useState } from 'react';
function App() {
    const [count, setCount] = useState(0);
    const increment = () => setCount(count + 1);
    const decrement = () => setCount(count - 1);
    return (
        <div>
            <h1>Count: {count}</h1> {/* Display the current count */}
            <button onClick={increment}>Increment</button> {/* Increment the count */}
            <button onClick={decrement}>Decrement</button> {/* Decrement the count */}
        </div>
    );
}
export default App;
```

REACT HOOKS – CONTEXT HOOKS

- The `useContext` hook in React allows functional components to access context values directly, without the need to manually pass props down through the component tree:

```
const contextValue = useContext(MyContext);
```

- The `useContext` hook takes a context object (`MyContext`) as an argument and returns the current value of that context.
- The `contextValue` will hold the value provided by the nearest `<MyContext.Provider>` in the component tree.

REACT HOOKS – CONTEXT HOOKS

```
import { createContext, useContext, useState }
from 'react';

const ThemeContext = createContext();

function App() {
    const [theme, setTheme] = useState("light");

    const toggleTheme = () => {
        setTheme((prevTheme) => (prevTheme ===
"light" ? "dark" : "light"));
    };
```

```
    return (
        <ThemeContext.Provider value={theme}>
            <div>
                <h1>Current Theme: {theme}</h1>
                <button onClick={toggleTheme}>
Toggle Theme</button>
                <ThemeDisplay />
            </div>
        </ThemeContext.Provider>
    );
}

function ThemeDisplay() {
    const theme = useContext(ThemeContext);

    return <h2>Theme from Context: {theme}</h2>;
}

export default App;
```

REACT HOOKS – EFFECT HOOKS

- Effect hooks, specifically `useEffect`, `useLayoutEffect`, and `useInsertionEffect`, enable functional components to handle side effects in a more efficient and modular way.
- `useEffect:` The `useEffect` hook in React is used to handle side effects in functional components. It allows you to perform actions such as data fetching, DOM manipulation, and setting up subscriptions, which are typically handled in lifecycle methods like `componentDidMount` or `componentDidUpdate` in class components.

```
useEffect(() => {
    // Side effect logic here
}, [dependencies]);
```

- `useEffect(() => { ... }, [dependencies]);` runs side effects after rendering.
- The effect runs based on changes in the specified dependencies.

REACT HOOKS – EFFECT HOOKS

- `useLayoutEffect:` The `useLayoutEffect` is used when we need to measure or manipulate the lawet before the browser paints, ensuring smooth transitions and no flickering.

```
useLayoutEffect(() => {
  // Logic to manipulate layout or measure DOM elements
}, [dependencies]);
```

- `useInsertionEffect:` The `useInsertionEffect` is designed for injecting styles early, especially useful for server-side rendering (SSR) or styling libraries, ensuring styles are in place before the component is rendered visually.

```
useInsertionEffect(() => {
    // Logic to inject styles or manipulate stylesheets
}, [dependencies]);
```

REACT HOOKS – EFFECT HOOKS

```
import { useState, useEffect } from 'react';

function App() {
    const [count, setCount] = useState(0);

    useEffect(() => {
        document.title = `Count: ${count}`;
        console.log(`Effect ran. Count is:
${count}`);

        return () => {
            console.log("Cleanup for previous
effect");
            document.title = "React App";
        };
    }, [count]);
```

```
    return (
        <div>
            <h1>Count: {count}</h1>
            <button onClick={() => setCount(count
+ 1)}>Increment Count</button>
        </div>
    );
}

export default App;
```

REACT HOOKS – PERFORMANCE HOOKS

- Performance Hooks in React, like `useMemo` and `useCallback`, are used to optimize performance by avoiding unnecessary re-renders or recalculations.
- `useMemo`: `useMemo` is a React hook that memorizes the result of an expensive calculation, preventing it from being recalculated on every render unless its dependencies change.
- This is particularly useful when we have a computation that is expensive in terms of performance, and we want to avoid recalculating it on every render cycle.

```
const memorizeValue = useMemo(() => computeExpensiveValue(a, b), [a, b]);
```

REACT HOOKS – PERFORMANCE HOOKS

- `useCallback:` The `useCallback` is a React hook that helps to memorize functions, ensuring that a function is not redefined on every render unless its dependencies change. This is particularly useful when passing functions as props to child components, as it prevents unnecessary re-renders of those child components.

```
const memorizeCallback = useCallback(() => { doSomething(a, b); }, [a, b]);
```

- `useMemo` caches the computed value (num * 2), recalculating it only when num changes.
- This prevents unnecessary calculations on every render.

REACT HOOKS – PERFORMANCE HOOKS

```
import { useState, useMemo } from 'react';

function App() {
    const [count, setCount] = useState(0);
    const [text, setText] = useState("");

    const expensiveCalculation = useMemo(() => {
        console.log("Expensive calculation...");
        return count * 2;
    }, [count]);

    return (
        <div>
            <h1>Count: {count}</h1>
            <h2>Expensive Calculation:
{expensiveCalculation}</h2>
            <button onClick={() => setCount(count
+ 1)}>Increment Count</button>
            <input
                type="text"
                value={text}
                onChange={(e) =>
setText(e.target.value)}
                placeholder="Type something"
            />
        </div>
    );
}

export default App;
```

REACT HOOKS – CUSTOM HOOKS

- Custom Hooks are user-defined functions that encapsulate reusable logic. They enhance code reusability and readability by sharing behavior between components.

```
import { useState, useEffect } from 'react';
function useWidth() {
    const [width, setWidth] = useState(window.innerWidth);
    useEffect(() => {
        const handleResize = () => setWidth(window.innerWidth);
        window.addEventListener("resize", handleResize);
        return () => window.removeEventListener("resize", handleResize);
    }, []);
    return width;
}
export default useWidth;
```

REACT HOOKS – CUSTOM HOOKS

```
import React from 'react';
import useWidth from './useWidth';
function App() {
    const width = useWidth();
    return <h1>Window Width: {width}px</h1>;
}
export default App;
```

- The custom hook `useWidth` encapsulates the logic for tracking the window's width.
- It reduces redundancy by reusing the logic across components.

REACT EVENTS

- In React, events are actions that occur within an application, such as **clicking a button**, **typing in a text field**, or **moving the mouse**.
- React provides an efficient way to handle these actions using its event system.
- Event handlers like onClick, onChange, and onSubmit are used to capture and respond to these events.

```
<element onEvent={handlerFunction} />
```

- **element:** The JSX element where the event is triggered (e.g., <button>, <input>, etc.).
- **onEvent:** The event name in camelCase (e.g., onClick, onChange).
- **handlerFunction:** The function that handles the event when it occurs.

REACT EVENTS

- React provides a variety of built-in event handlers that we can use to handle different user interactions:

React Event	Description
onClick	This event is used to detect mouse clicks in the user interface.
onChange	This event is used to detect a change in the input field in the user interface.
onSubmit	This event triggers when a form is submitted and is commonly used to handle form data while preventing the page from reloading.
onKeyDown	This event occurs when the user presses any key from the keyboard.
onKeyUp	This event occurs when the user releases any key from the keyboard.
onMouseEnter	This event occurs when the mouse enters the boundary of the element.

REACT EVENTS

Handling Events in React

1. **Adding Event Handlers**

- In React, event handlers are added directly to elements via JSX attributes.
- React uses the camelCase convention for event names, which differs from the standard HTML event names (e.g., onClick instead of onclick).

REACT EVENTS

```
import React, { Component } from 'react';

class App extends Component {
    constructor(props) {
        super(props);
        this.state = {
            message: 'Hello, welcome to React!',
        };

        this.handleClick = this.handleClick.bind(this);
    }


    handleClick() {

        this.setState({
            message: 'You clicked the button!',
        });
    }
```

```
    render() {
        return (
            <div>
                <h1>{this.state.message}</h1>
                {/* Add event handler to the button */}
                <button onClick={this.handleClick}>Click
Me</button>
            </div>
        );
    }
}

export default App;
```

REACT EVENTS

2. **Reading Props in Event Handlers**

- In React, event handlers often need access to the props passed from parent components.
- This is useful when the event handler needs to perform actions based on the data provided via props.

```
// App.js
import React, { Component } from 'react';
import Child from './Child';
import './App.css';
class Parent extends Component {
    render() {
        return (
            <div className="parent-container">
                <h1>Parent Component</h1>
                <Child greeting="Hello from Parent!" />
            </div>
        );
    }
}
export default Parent;
```

REACT EVENTS

```
// Child.js
import React, { Component } from 'react';
class Child extends Component {
    handleClick = () => {
        alert(this.props.greeting);
    };
    render() {
        return (
            <div className="child-container">
                <h2>Child Component</h2>
                <button onClick={this.handleClick}>Click to See Greeting</button>
            </div>
        );
    }
}
export default Child;
```

REACT EVENTS

```
// App.css
body {
    margin: 0;
    padding: 0;
    display: flex;
    align-items: flex-start;
    height: 100vh;
    background-color: #f0f0f0;
}

.parent-container {
    text-align: center;
}

.child-container {
    display: inline-block;
    margin-top: 20px;
    padding: 20px;
    background-color: #fff;
    border: 1px solid #ddd;
    box-shadow: 0 4px 8px rgba(0, 0, 0,
0.1);
}
```

```
button {
    padding: 10px 20px;
    background-color: #007BFF;
    color: white;
    border: none;
    cursor: pointer;
    font-size: 16px;
}

button:hover {
    background-color: #0056b3;
}
```

REACT EVENTS

3. **Passing Event Handlers as Props**

- Event handlers can be passed down to child components as props.
- This allows child components to communicate back to the parent and trigger actions in the parent when events occur.

```
// App.js
import React, { Component } from 'react';
import Child from './Child';

class Parent extends Component {

    handleClick = () => {
        alert("Button clicked in Child component!");
    };

    render() {
        const containerStyle = {
            display: "flex",
            justifyContent: "center",
            alignItems: "flex-start",
            height: "100vh",
            margin: "0",
        };
```

```
 return (
            <div style={containerStyle}>
                <div>
                    <h1>Parent Component</h1>
                    {/* Passing the event handler as a
prop to the Child component */}
                    <Child
onClickHandler={this.handleClick} />
                </div>
            </div>
        );
    }
}

export default Parent;
```

REACT EVENTS

```
// Child.js
import React from 'react';

function Child({ onClickHandler }) {
    const buttonStyle = {
        padding: "12px 24px",
        fontSize: "16px",
        color: "white",
        backgroundColor: "#007bff",
        border: "none",
        borderRadius: "4px",
        cursor: "pointer",
        transition: "background-color 0.3s",
    };

    const buttonHoverStyle = {
        backgroundColor: "#0056b3",
    };
```

```
 return (
        <div>
            <h2>Child Component</h2>
            {/* Button to call the event handler
*/}
            <button
                onClick={onClickHandler}
                style={buttonStyle}
                onMouseOver={(e) =>
(e.target.style.backgroundColor =
buttonHoverStyle.backgroundColor)}
                onMouseOut={(e) =>
(e.target.style.backgroundColor = "#007bff")}
            >
                Click Me!
            </button>
        </div>
    );
}

export default Child;
```

REACT EVENTS

4. **Naming Event Handler Props**

- In React, it's common to name event handler props according to the event being handled.

```
import React from "react";
function Button({ onClickHandler }) {
    return <button onClick={onClickHandler}>Click Me</button>;
}
function Parent() {
    const handleClick = () => {
        alert("Button clicked!");
    };
    return <Button onClickHandler={handleClick} />;
}
export default Parent;
```

DIFFERENCE BETWEEN HTML DOM AND REACT DOM

Feature	HTML DOM	React DOM
Nature	Represents the structure of the HTML document.	Represents the virtual representation of the UI.
Updates	Updates the actual DOM directly after every change.	Updates the virtual DOM first, then selectively updates the real DOM.
Performance	Can lead to performance issues with frequent direct updates	Optimized performance using a virtual DOM and efficient reconciliation.
Event Handling	Requires a full re-render of the page or element.	Events are attached to the virtual DOM and handled by React's synthetic event system.
Data Binding	Events are directly attached to DOM elements.	React uses state and props to manage and bind data between components.
DOM Representation	The DOM is a tree structure where each element is a node.	React DOM uses a virtual DOM, a lightweight copy of the real DOM for efficient updates.